

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский Нижегородский
государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Центр прикладных информационных технологий

Направление подготовки: «Фундаментальная информатика и
информационные технологии»

Профиль подготовки: «Инженерия программного обеспечения»

Отчет по практической работе
на тему:

«Алгоритм Дейкстры на 3-куче и 15-куче»

Выполнил:

студент группы 3822Б1ФИ1

_____ Чистов А. Д.

Проверил:

_____ Уткин Г. В.

Нижний Новгород
2024 г.

Содержание

Введение

Одной из центральных задач теории графов является нахождение кратчайших путей в графах, что имеет широкое практическое применение в задачах маршрутизации, логистики, сетевого планирования и моделирования. Алгоритм Дейкстры, предложенный Эдсгером Дейкстрой в 1959 году, стал классическим подходом для решения этой задачи на графах с ненулевыми положительными весами рёбер [?]. Алгоритм Дейкстры позволяет находить кратчайший путь от одной вершины графа до всех остальных, и основой его эффективности является использование структур данных, таких как куча, для оптимизации извлечения минимального значения.

Обычно в алгоритме Дейкстры используется двоичная куча, обеспечивающая хорошую производительность для стандартных задач. Однако для улучшения производительности при работе с большими графами и для различных практических применений может использоваться D-арная куча (например, 3-куча, 15-куча и др.), позволяющая сократить количество операций по сравнению с двоичной кучей за счёт уменьшения высоты дерева. В частности, 3-куча и 15-куча могут быть полезны для ускорения алгоритма в зависимости от структуры графа и особенностей доступа к памяти.

Цели и задачи данной работы:

1. Исследовать тематическую литературу.
2. Реализовать класс DHeap.
3. Реализовать алгоритм Дейкстры.
4. Провести экспериментальное сравнение производительности и вычислительной эффективности алгоритма на различных типах графов.
5. Сделать выводы о проделанной работе.

1 Описание алгоритмов

1.1 Описание D-кучи

Класс *DHeap* реализует d -арную кучу, где d — количество дочерних узлов для каждого узла.

- Конструктор принимает целое значение d и проверяет, что оно больше или равно 2. Если $d < 2$, генерируется исключение.

```
1 DHeap::DHeap(int d) : d_(d) {  
2     if (d_ < 2) throw std::runtime_error("Degree_of_heap_must_be_at_least_2.  
3     ");  
4 }
```

- **Insert** вставляет новый элемент в кучу. Элемент добавляется в конец вектора, после чего вызывается функция **SiftUp** для поддержания свойств кучи.

```
1 void DHeap::Insert(int key, int value) {  
2     v_.push_back({ key, value });  
3     SiftUp(v_.size() - 1);  
4 }
```

- **ExtractMin** извлекает минимальный элемент из кучи. Если куча пуста, генерируется исключение. Минимальный элемент сохраняется, последний элемент вектора перемещается на место корня, после чего вызывается **SiftDown**.

```
1 pair<int, int> DHeap::ExtractMin() {  
2     if (v_.IsEmpty()) throw std::runtime_error("Heap_is_empty");  
3  
4     pair<int, int> min_element(v_[0]);  
5     v_[0] = v_.back();  
6     v_.pop_back();  
7     if (!v_.IsEmpty()) SiftDown(0);  
8  
9     return min_element;  
10 }
```

- **IsEmpty** проверяет, пуста ли куча.

```
1 bool DHeap::IsEmpty() const {  
2     return v_.IsEmpty();  
3 }
```

- **SiftDown** восстанавливает свойства кучи, начиная с узла i . Перемещает элемент вниз по дереву, пока он не станет больше своих дочерних элементов.

```

1 void DHeap::SiftDown(size_t i) {
2     int key0 = v_[i].first;
3     int value0 = v_[i].second;
4     size_t c = MinChild(i);
5
6     while (c != i && value0 > v_[c].second) {
7         v_[i] = v_[c];
8         i = c;
9         c = MinChild(i);
10    }
11    v_[i] = { key0, value0 };
12 }

```

- **SiftUp** восстанавливает свойства кучи, начиная с узла i . Перемещает элемент вверх по дереву, пока он не станет меньше своих родительских узлов.

```

1 void DHeap::SiftUp(size_t i) {
2     int key0 = v_[i].first;
3     int value0 = v_[i].second;
4     size_t p = Father(i);
5
6     while (i != 0 && v_[p].second > value0) {
7         v_[i] = v_[p];
8         i = p;
9         p = Father(i);
10    }
11    v_[i] = { key0, value0 };
12 }

```

- **MinChild** находит индекс минимального дочернего элемента для узла i . Если у узла нет дочерних элементов, возвращается индекс самого узла.

```

1 size_t DHeap::MinChild(size_t i) {
2     size_t first = FirstChild(i);
3     if (first == -1) return i;
4
5     size_t last = LastChild(i);
6     size_t min_idx = first;
7
8     for (size_t j = first + 1; j <= last; ++j) {
9         if (v_[j].second < v_[min_idx].second) {

```

```

10         min_idx = j;
11     }
12 }
13 return min_idx;
14 }

```

- **LastChild** возвращает индекс последнего дочернего элемента для узла i . Если у узла нет дочерних элементов, возвращается -1.

```

1 size_t DHeap::LastChild(size_t i) const {
2     size_t first = FirstChild(i);
3     return (first == -1) ? -1 : std::min(first + d_ - 1, v_.size() - 1);
4 }

```

- **FirstChild** возвращает индекс первого дочернего элемента для узла i . Если узел не имеет дочерних элементов, возвращается -1.

```

1 size_t DHeap::FirstChild(size_t i) const {
2     size_t k = d_ * i + 1;
3     return (k < v_.size()) ? k : -1;
4 }

```

- **Father** возвращает индекс родительского узла для узла i .

```

1 size_t DHeap::Father(size_t i) const {
2     return (i - 1) / d_;
3 }

```

1.2 Описание алгоритма Дейкстры

Алгоритм состоит из двух основных компонентов: **генерация графа** и **алгоритм Дейкстры**.

Функция `generateGraph(int n, int m, int q, int r)` генерирует случайный неориентированный граф с n вершинами и до m ребер. Вес ребер присваиваются случайным образом в диапазоне от q до r .

Максимальное количество ребер ограничено формулой $\frac{n(n-1)}{2}$, которая представляет количество ребер в полном графе. Двумерный вектор `was` используется для отслеживания существующих ребер, чтобы избежать дублирования. Ребра добавляются в список смежности `adj_list` до достижения желаемого количества ребер.

```

1 m = min(n * (n - 1) / 2, m);
2 vector<vector<pair<int, int>>> adj_list(n);
3 std::random_device rd;
4 std::mt19937 gen(rd());
5 std::uniform_int_distribution<> dist(q, r);
6 vector<vector<bool>> was(n, vector<bool>(n, false));
7 int edge_count = 0;
8
9 while (edge_count < m) {
10     int i = rand() % n;
11     int j = rand() % n;
12     if (i != j && !was[i][j]) {
13         int weight = dist(gen);
14         adj_list[i].push_back({ j, weight });
15         adj_list[j].push_back({ i, weight });
16         was[i][j] = was[j][i] = true;
17         ++edge_count;
18     }
19 }

```

Функция реализует алгоритм Дейкстры для нахождения кратчайшего пути от стартовой вершины ко всем другим вершинам графа.

Алгоритм инициализирует расстояния до бесконечности (INT_MAX) и устанавливает расстояние до стартовой вершины равным нулю. Приоритетная очередь, представляемая Д-ариНеар (DHeap), используется для эффективного извлечения вершины с минимальным расстоянием. Алгоритм итеративно обрабатывает вершины, обновляя расстояния до их соседей, если найден более короткий путь.

```

1 vector<bool> processed(n, false);
2 vector<int> index(n);
3 vector<int> name(n);
4
5 for (int i = 0; i < n; ++i) {
6     up[i] = -1;
7     dist[i] = INT_MAX;
8     index[i] = i;
9     name[i] = i;
10 }
11
12 dist[start] = 0;
13 up[start] = start;
14 DHeap dHeap(d);
15 dHeap.Insert(start, dist[start]);
16
17 while (!dHeap.IsEmpty()) {
18     pair<int, int> minPair = dHeap.ExtractMin();

```

```

19     int i = minPair.first;
20     int currentDist = minPair.second;
21
22     if (processed[i]) continue;
23     processed[i] = true;
24
25     vector<pair<int, int>> edge = graph[i];
26     for (int k = 0; k < edge.size(); k++) {
27         int j = edge[k].first;
28         int newDist = edge[k].second + currentDist;
29
30         if (newDist < dist[j]) {
31             dist[j] = newDist;
32             up[j] = i;
33             dHeap.Insert(j, dist[j]);
34         }
35     }
36 }

```

Время выполнения алгоритма Дейкстры измеряется с помощью высокоточных часов из библиотеки `chrono`. Время, затраченное на выполнение алгоритма, выводится в конце.

2 Описание программной реализации

2.1 Описание класса DHeap

```
class DHeap {
private:
    int d_;
    vector<pair<int, int>> v_;

    size_t MinChild(size_t i);
    size_t LastChild(size_t i) const;
    size_t FirstChild(size_t i) const;
    size_t Father(size_t i) const;
    void SiftDown(size_t i);
    void SiftUp(size_t i);
public:
    DHeap(int d);
    void Insert(int key, int value);
    pair<int, int> ExtractMin();
    bool IsEmpty() const;
};
```

Поля

- `d_` – количество детей для каждого узла в куче.
- `v_` – вектор пар, хранящий ключи и значения в куче.

Методы

- `DHeap(int d);`
 - **Назначение:** Конструктор, создающий кучу с заданным числом детей для каждого узла.
 - **Входные параметры:** `d` – количество детей для каждого узла.
 - **Выходные параметры:** отсутствуют.
- `void Insert(int key, int value);`
 - **Назначение:** Вставляет новый элемент в кучу.
 - **Входные параметры:** `key` – ключ элемента, `value` – значение элемента.

- **Выходные параметры:** отсутствуют.
- `pair<int, int> ExtractMin();`
 - **Назначение:** Извлекает элемент с минимальным ключом из кучи.
 - **Входные параметры:** отсутствуют.
 - **Выходные параметры:** пара (`key`, `value`) минимального элемента.
- `bool IsEmpty() const;`
 - **Назначение:** Проверяет, пуста ли куча.
 - **Входные параметры:** отсутствуют.
 - **Выходные параметры:** `true`, если куча пуста, иначе `false`.
- `size_t MinChild(size_t i);`
 - **Назначение:** Находит индекс минимального дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс минимального дочернего узла.
- `size_t LastChild(size_t i) const;`
 - **Назначение:** Находит индекс последнего дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс последнего дочернего узла.
- `size_t FirstChild(size_t i) const;`
 - **Назначение:** Находит индекс первого дочернего узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс первого дочернего узла.
- `size_t Father(size_t i) const;`
 - **Назначение:** Находит индекс родительского узла для узла с индексом `i`.
 - **Входные параметры:** `i` – индекс узла.
 - **Выходные параметры:** индекс родительского узла.
- `void SiftDown(size_t i);`
 - **Назначение:** Восстанавливает свойства кучи, двигая узел вниз по дереву.

- **Входные параметры:** i – индекс узла.
- **Выходные параметры:** отсутствуют.
- `void SiftUp(size_t i);`
 - **Назначение:** Восстанавливает свойства кучи, двигая узел вверх по дереву.
 - **Входные параметры:** i – индекс узла.
 - **Выходные параметры:** отсутствуют.

2.2 Описание функций

Функция для создания графа

```
vector<vector<pair<int, int>>> generateGraph(int n, int m, int q, int r) {
    m = min(n * (n - 1) / 2, m);
    vector<vector<pair<int, int>>> adj_list(n);

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dist(q, r);

    vector<vector<bool>> was(n, vector<bool>(n, false));

    int edge_count = 0;

    while (edge_count < m) {
        int i = rand() % n;
        int j = rand() % n;
        if (i != j && !was[i][j]) {
            int weight = dist(gen);

            adj_list[i].push_back({ j, weight });
            adj_list[j].push_back({ i, weight });

            was[i][j] = was[j][i] = true;

            ++edge_count;
        }
    }
    return adj_list;
}
```

```
}
```

Назначение:

Генерация случайного графа с указанным числом узлов и рёбер, где рёбра имеют случайные веса.

Входные параметры:

- `int n` – количество узлов в графе.
- `int m` – максимальное количество рёбер.
- `int q` – минимальный вес рёбер.
- `int r` – максимальный вес рёбер.

Выходные параметры:

- Возвращает вектор векторов пар, представляющий список смежности графа, где каждая пара содержит узел и вес рёбра.

Алгоритм Дейкстры

```
double dijkstra(vector<int>& dist, vector<int>& up,
vector<vector<pair<int, int>>> graph, int n, int d, int start) {
    vector<bool> processed(n, false);
    vector<int> index(n);
    vector<int> name(n);

    for (int i = 0; i < n; ++i) {
        up[i] = -1;
        dist[i] = INT_MAX;
        index[i] = i;
        name[i] = i;
    }

    dist[start] = 0;
    up[start] = start;
    DHeap dHeap(d);
    dHeap.Insert(start, dist[start]);

    auto begin = std::chrono::high_resolution_clock::now();
```

```

while (!dHeap.IsEmpty()) {
    pair<int, int> minPair = dHeap.ExtractMin();
    int i = minPair.first;
    int currentDist = minPair.second;

    if (processed[i]) continue;
    processed[i] = true;

    vector<pair<int, int>> edge = graph[i];
    for (int k = 0; k < edge.size(); k++) {
        int j = edge[k].first;
        int newDist = edge[k].second + currentDist;

        if (newDist < dist[j]) {
            dist[j] = newDist;
            up[j] = i;
            dHeap.Insert(j, dist[j]);
        }
    }
}

auto end = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> executionTime = end - begin;
std::cout << "Execution time: " << executionTime.count() << " seconds\n";

return executionTime.count();
}

```

Назначение:

Реализация алгоритма Дейкстры для поиска кратчайшего пути в графе.

Входные параметры:

- `vector<int>& dist` – вектор, в который будут записаны кратчайшие расстояния от начального узла до всех других узлов.
- `vector<int>& up` – вектор, который будет хранить предшественников для каждого узла.

- `vector<vector<pair<int, int>>> graph` – список смежности графа.
- `int n` – количество узлов в графе.
- `int d` – степень DHeap.
- `int start` – начальный узел для поиска кратчайших путей.

Выходные параметры:

- Возвращает время выполнения алгоритма в секундах.

3 Анализ графиков

Ниже показано изображение:

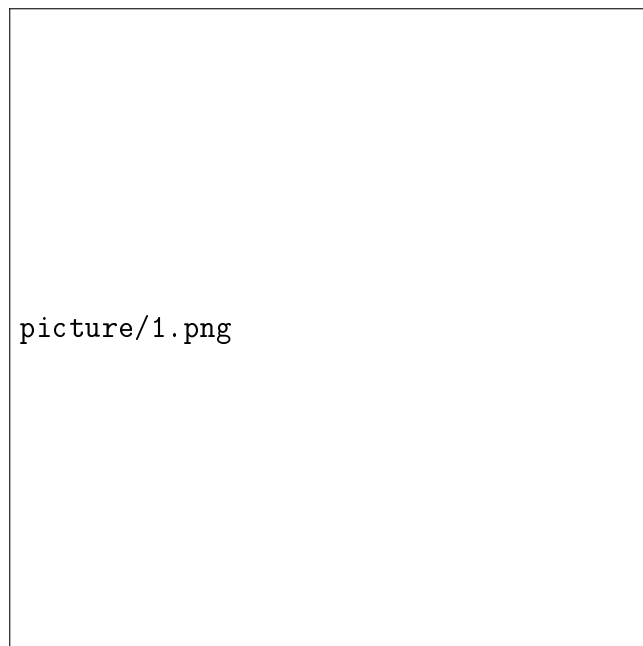


Рис. 1: Пример изображения

Заключение

В ходе проведенного исследования был рассмотрен алгоритм Дейкстры на d -куче, с фокусом на реализацию с параметрами $d = 3$ и $d = 15$. Анализ полученных результатов, а также построенные графики, показали, что при увеличении параметра d существенного различия в производительности алгоритма не наблюдается. Незначительные отклонения в результатах можно объяснить погрешностями, связанными с особенностями работы операционной системы Windows, что потенциально влияет на время выполнения и управляет распределением ресурсов во время исполнения алгоритма.

Во время исследования были достигнуты следующие цели и выполнены поставленные задачи:

1. Изучены особенности алгоритма Дейкстры и его модификации для d -куч.
2. Реализованы и протестированы алгоритмы на d -кучах с параметрами $d = 3$ и $d = 15$.
3. Проведен сравнительный анализ времени выполнения алгоритмов и построены графики зависимости производительности от параметров.

Полученные результаты позволяют заключить, что увеличение значения параметра d в d -куче для реализации алгоритма Дейкстры не оказывает значительного влияния на эффективность. Это объясняется тем, что при больших значениях d по-прежнему доминирует природа самого алгоритма поиска кратчайших путей, где основное время выполнения тратится на операции извлечения и обновления приоритетов.

В заключение, можно отметить, что практическое применение d -кучи с большими значениями d может оказаться нецелесообразным, так как увеличивается сложность структуры данных без значительных улучшений в скорости работы алгоритма. Тем не менее, данное исследование позволило углубить понимание свойств d -куч в контексте алгоритма Дейкстры и расширить знания о том, как особенности архитектуры системы могут оказывать влияние на результаты вычислений.

Список литературы

1. Алгоритм Дейкстры // Википедия: свободная энциклопедия.
URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
2. Кормен Т.Х., Лейзерсон Ч.Е., Ривест Р.Л., Штайн К. Алгоритмы: Построение и анализ / Т.Х. Кормен, Ч.Е. Лейзерсон, Р.Л. Ривест, К. Штайн. — 3-е изд. — М.: Вильямс, 2011.
3. Скиен С. Алгоритмы. Руководство по разработке / С. Скиен. — М.: Вильямс, 2006. — 800 с.: ил. ISBN 978-5-8459-0805-7.

Приложения

Приложение А. Реализация DHeap

Листинг 1: Приложение А. Реализация DHeap

```
1 class DHeap {
2 public:
3     DHeap(int d);
4     void Insert(int key, int value);
5     std::pair<int, int> ExtractMin();
6     bool IsEmpty() const;
7
8 private:
9     size_t MinChild(size_t i);
10    size_t LastChild(size_t i) const;
11    size_t FirstChild(size_t i) const;
12    size_t Father(size_t i) const;
13    void SiftDown(size_t i);
14    void SiftUp(size_t i);
15
16    int d_;
17    std::vector<std::pair<int, int>> v_;
18 };
19
20 size_t DHeap::MinChild(size_t i) {
21     size_t first = FirstChild(i);
22     if (first == -1) return i;
23
24     size_t last = LastChild(i);
25     size_t min_idx = first;
26
27     for (size_t j = first + 1; j <= last; ++j) {
28         if (v_[j].second < v_[min_idx].second) {
29             min_idx = j;
30         }
31     }
32     return min_idx;
33 }
34
35 size_t DHeap::LastChild(size_t i) const {
36     size_t first = FirstChild(i);
37     return (first == -1) ? -1 : std::min(first + d_ - 1, v_.size() - 1);
38 }
39
40 size_t DHeap::FirstChild(size_t i) const {
41     size_t k = d_ * i + 1;
```

```

42     return (k < v_.size()) ? k : -1;
43 }
44
45 size_t DHeap::Father(size_t i) const {
46     return (i - 1) / d_;
47 }
48
49 void DHeap::SiftDown(size_t i) {
50     int key0 = v_[i].first;
51     int value0 = v_[i].second;
52     size_t c = MinChild(i);
53
54     while (c != i && value0 > v_[c].second) {
55         v_[i] = v_[c];
56         i = c;
57         c = MinChild(i);
58     }
59     v_[i] = { key0, value0 };
60 }
61
62 void DHeap::SiftUp(size_t i) {
63     int key0 = v_[i].first;
64     int value0 = v_[i].second;
65     size_t p = Father(i);
66
67     while (i != 0 && v_[p].second > value0) {
68         v_[i] = v_[p];
69         i = p;
70         p = Father(i);
71     }
72     v_[i] = { key0, value0 };
73 }
74
75 DHeap::DHeap(int d) : d_(d) {
76     if (d_ < 2) throw std::runtime_error("Degree of heap must be at least 2.");
77 }
78
79 void DHeap::Insert(int key, int value) {
80     v_.push_back({ key, value });
81     SiftUp(v_.size() - 1);
82 }
83
84 std::pair<int, int> DHeap::ExtractMin() {
85     if (v_.empty()) throw std::runtime_error("Heap is empty");
86
87     std::pair<int, int> min_element(v_[0]);

```

```

88     v_[0] = v_.back();
89     v_.pop_back();
90     if (!v_.empty()) SiftDown(0);
91
92     return min_element;
93 }
94
95 bool DHeap::IsEmpty() const {
96     return v_.empty();
97 }

```

Приложение Б. Реализация Дейкстры

Листинг 2: Приложение Б. Реализация Дейкстры

```

1  vector<vector<pair<int, int>>> generateGraph(int n, int m, int q, int r) {
2      m = min(n * (n - 1) / 2, m);
3      vector<vector<pair<int, int>>> adj_list(n);
4
5      std::random_device rd;
6      std::mt19937 gen(rd());
7      std::uniform_int_distribution<> dist(q, r);
8
9      vector<vector<bool>> was(n, vector<bool>(n, false));
10     int edge_count = 0;
11
12     while (edge_count < m) {
13         int i = rand() % n;
14         int j = rand() % n;
15         if (i != j && !was[i][j]) {
16             int weight = dist(gen);
17
18             adj_list[i].push_back({ j, weight });
19             adj_list[j].push_back({ i, weight });
20
21             was[i][j] = was[j][i] = true;
22             ++edge_count;
23         }
24     }
25     return adj_list;
26 }
27
28 double dijkstra(vector<int>& dist, vector<int>& up, vector<vector<pair<int,
29     int>>> graph, int n, int d, int start) {
30     vector<bool> processed(n, false);
31     vector<int> index(n);

```

```

31     vector<int> name(n);
32
33     for (int i = 0; i < n; ++i) {
34         up[i] = -1;
35         dist[i] = INT_MAX;
36         index[i] = i;
37         name[i] = i;
38     }
39
40     dist[start] = 0;
41     up[start] = start;
42     DHeap dHeap(d);
43     dHeap.Insert(start, dist[start]);
44
45     auto begin = std::chrono::high_resolution_clock::now();
46
47     while (!dHeap.IsEmpty()) {
48         pair<int, int> minPair = dHeap.ExtractMin();
49         int i = minPair.first;
50         int currentDist = minPair.second;
51
52         if (processed[i]) continue;
53         processed[i] = true;
54
55         vector<pair<int, int>> edge = graph[i];
56         for (int k = 0; k < edge.size(); k++) {
57             int j = edge[k].first;
58             int newDist = edge[k].second + currentDist;
59
60             if (newDist < dist[j]) {
61                 dist[j] = newDist;
62                 up[j] = i;
63                 dHeap.Insert(j, dist[j]);
64             }
65         }
66     }
67
68     auto end = std::chrono::high_resolution_clock::now();
69     std::chrono::duration<double> executionTime = end - begin;
70     std::cout << "Execution time: " << executionTime.count() << " seconds\n"
71     ;
72
73     return executionTime.count();
74 }

```